

Information Collection, Classification, and Knowledge Base Development

Mfano Bora Africa Chatbot Project Status Report

Introduction

The primary objective of this project is to develop an intelligent, machine learning-based, web-integrated chatbot that serves as a frontline digital assistant for Mfano Bora Africa Ltd. To ensure the chatbot delivers highly accurate, factual responses and eliminates model hallucinations, its underlying knowledge base must be systematically constructed using verified sources.

This report summarises the practical implementation steps undertaken to fulfil Task 3 (Information Collection), Task 4 (Information Classification), and Task 5 (Knowledge Base Development)[cite: 2, 4]. It outlines the concrete actions taken to establish the project's infrastructure, specifically detailing the creation of the GitHub repository, the configuration of the Supabase PostgreSQL database, and the deployment of the automated data pipeline.

Action Steps Summary

1. Version Control and GitHub Repository Setup

To establish a robust development environment, the project's version control infrastructure was configured:

- **Repository Initialisation:** A local project folder named `mfano-bora-chatbot-system` was created and linked to the remote GitHub repository located at <https://github.com/Kenjin32/icon/mfano-bora-chatbot-system.git>
- **Security Configurations:** A `.gitignore` file was implemented prior to the first commit to ensure that environment secrets and virtual environment files are never pushed to the public repository.
- **IDE Integration:** The project was integrated with VSCodium to manage source control and streamline the development workflow.

2. Supabase Database Configuration (Task 5)

To provide a structured environment for the chatbot's knowledge base, a vector-capable database was set up:

- **Project Creation:** The Supabase database project was successfully established and is accessible at <https://lurtyfvgdqzxlrohjgs.supabase.co>
- **Schema Initialisation:** Using the `database/init\_schema.sql` script, the `pgvector` extension was enabled to allow for semantic similarity searches.

- **Table Structuring:** The ``knowledge_base`` table was created to store the data, configured with specific columns for the source URL, classification category, cleaned text content, and a 384-dimensional vector embedding.

3. Python Environment and Pipeline Preparation

The foundation for the automated data processing scripts was laid out:

- **Virtual Environment:** A Python virtual environment (`.venv``) was created and activated to isolate project dependencies.
- **Dependencies:** Essential libraries, including ``beautifulsoup4`` for web scraping, ``sentence-transformers`` for vectorisation, and ``supabase`` for database connectivity, were installed via a ``requirements.txt`` file.
- **Secret Management:** Supabase API credentials (URL and service role key) were securely stored locally within a `.env`` file.

4. Automated Information Collection & Classification (Tasks 3 & 4)

An automated web scraper was developed to programmatically gather and categorise verified data:

- **Data Extraction & Cleaning:** A Python script (``ai-pipeline/data-cleaning/scraper.py``) was implemented to navigate the verified primary sources, extract raw HTML, strip out duplicate tags, and break the cleaned text into structured chunks.
- **Dynamic Classification:** Addressing Task 4, the scraper automatically assigns metadata tags to each chunk as it is processed. These tags include the information category, target user group, and update frequency, ensuring systematic classification.
- **Data Staging:** The classified chunks are temporarily saved as a JSON file in the ``ai-pipeline/knowledge-base/`` directory.

5. Vectorisation and Knowledge Base Ingestion (Task 5)

The final step involved converting the classified data into a machine-readable format for the chatbot:

- **Embedding Generation:** A vectorisation script (``ai-pipeline/embeddings/embed_and_store.py``) was developed to process the staged text chunks. It utilises a Hugging Face Sentence Transformer model (``all-MiniLM-L6-v2``) to convert the text into numerical vector embeddings.
- **Database Ingestion:** The script programmatically inserts the processed chunks, along with their corresponding vector embeddings and classification metadata, directly into the Supabase ``knowledge_base`` table. This completes the structured knowledge base required by the Retrieval-Augmented Generation (RAG) engine.